

אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Hatchala - Introduction to computer science in Java - Fall 26'-27'

Department of Computer Science and Information

Ben-Gurion University of the Negev

Based on both Dr. Assaf Zaritsky slides for "Intro to CS" (IS)
and slides for "Introduction to CS" (CS) courses at BGU

Lecture 1: First acquaintance with Java

Michal Shemesh

- Adjunct Lecturer in the CSI Faculty, CS Department since 2010.
- Graduated both degrees in Computer Science at BGU.
- Plans ahead.. there's a lot ☺

- Thought courses for first & second year students in CS.
- I'll be the head Lecturer (מרכזת) at your upcoming "Introduction to computer science" course (CIS faculty) this semester.

Welcome to BGU!
Before we start…

- Ground rules (for the program and the upcoming 3/4 years):
  - English – get used to it.
  - USE EXTERNAL SOURCES RESPONSIBLY-
    - Google / AI - use wisely!!!
  - Share knowledge!
  - Ask whenever/whatever – but try to figure out on your own first.
  - No cheating (plagiarism).

הקורס מבוא למדעי המחשב -לאן מועדות פנינו?

מטרת הקורס :הכרות ראשונית עם ליבת מדעי המחשב –
היבטים תאורטיים ומעשיים של פתרון בעיות על ידי שיטות חישוביות.

כיצד נגיע לשם?



בסיום הקורס

- ✓ **נכיר מושגי יסוד.**
- ✓ **נכיר בעיות חישוביות קלאסיות.**
- ✓ **נדע להכליל דוגמאות .**
- ✓ **נבחר אסטרטגיות פתרון לבעיות חדשות.**
- ✓ **ננסח פתרון לבעיה כאלגוריתם.**
- ✓ **נכתוב אלגוריתם בשפת המחשב Java.**

נרכוש הרגלי למידה נכונים -צריך ללמוד להיות סטודנט/ית.

נעזר בצוות הקורס -לא הביישן למד!

לא נתייאש ,גם כשקשה וגם כשלא נצליח.



נקדיש זמן רב לחזרה על החומר ועבודה עצמית על בעיות חדשות ברמות קושי שונות .

נראה מגוון בעיות ונפגוש אסטרטגיות פתרון עבור סוגים שונים של בעיות.

נפגוש מספר רב של נושאים מעולם מדעי המחשב.

Hatchala course objective

Students will feel comfortable approaching simple coding assignments.



We will learn to program in **Java**

Why Java?

- It teaches **strong programming fundamentals**
- It builds a **bridge to learning other languages**
- It's widely **used in academia and industry**
- Used in many CS1+CS2 courses in **top institutes**
- Used in Your "**Intro to Computer Science**" + "**Programming fundamentals in Java**" courses this upcoming semester (in following semesters as well…)



- It's an object-oriented language
- It's safe and strongly typed
- It's platform-independent
- It has a huge ecosystem
- etc…

Message from your peers

- Be active – Tons of resources out there
- Practice Practice Practice
- Semester Course is H – A – R – D !
- Blocking Course

Programing dependency

- Starting 2nd year – ~3 courses a semester will require programming
- You will use java also in your second and third semester – along the way, if another language is required, you will mostly learn it yourself…

After the 1st semester, Everyone Assumes you know how to code!

Administration

Course staff

Lecturer:

Michal Shemesh

Teaching Assistants:

Ron Franco

Daniel Dubinskey

Nadav Alali

Omer Aviram

Administration

Moodle: *http://moodle.bgu.ac.il*

Everything is there:

- All administrative information
- Staff emails
- Lectures slides
- Practical sessions slides
- Home Assignment
- Announcements

Make sure you are updated…

Lectures vs. Practical sessions

- In a standard classroom
- Purposes:
  - Practice topics presented in class
  - Acquire tools for problem solving
  - Learn from experienced coders.
  - Preparations for next class.
  - Learn practical tools and ideas.

- Lecture cover theoretical aspects with fewer examples.
- Lectures will be harder to understand, and it is **OK**
- What does **OK** means?

A personal note on future HW

It will take you a **lot of time** and effort

But

There is no other way to learn how to program



Assignments

- The assignments for the semester course will be published periodically during the semester.
- There are 5-6 programming assignments during the semester.
- This is to **encourage** you to **practice** before the start of the semester, and to introduce you to the level you are required to be at.

Cheating – relevant for the semester

- It is allowed (and encouraged) to talk, share ideas, and help friends. No code-sharing!
- Without coding and solving problems by your self you will probably will Not succeed.

Tentative syllabus
(we might not get it all done..)

- Java programming basics
  - Variables.
  - Arithmetic operators.
  - Conditional statements.
  - Loops.
  - Arrays
  - Functions.
  - Testing

Resources

- Course slides and codes are available
- Other resources:
  - ✓ The Java Tutorials
  - ✓ Oracle Java API
  - ✓ Google
- Online free courses, for example:
  - ✓ FreeCodeCamp - Free Java Courses for Beginners
  - ✓ Codecademy Learn Java course (Free)
  - ✓ Coursera - Computer Science: Programming with a Purpose – Princeton University

Questions?

Preface

- We assume no prior knowledge in programming
- However, students understand on different paces
- The only way to keep on track is to **practice**!

Technical aspects in recitations

- Java Development Kit (JDK) version 25
  not 26 which has just emerged!
- Integrated Development Environment (IDE) –

  IntelliJ (CIS course) OR eclipse (IE&M course)

Today's Agenda

- **Brief background to computation and programming**
- Java basics:
  - Variables
  - Arithmetic operators
  - Logic operators
  - Input/Output
  - Conditional statements



Programming languages basics

- A computer program is a sequence of text instructions that can be “understood" by a computer and executed
- A programming language is a machine-readable artificial language designed to express computations that can be performed by a computer

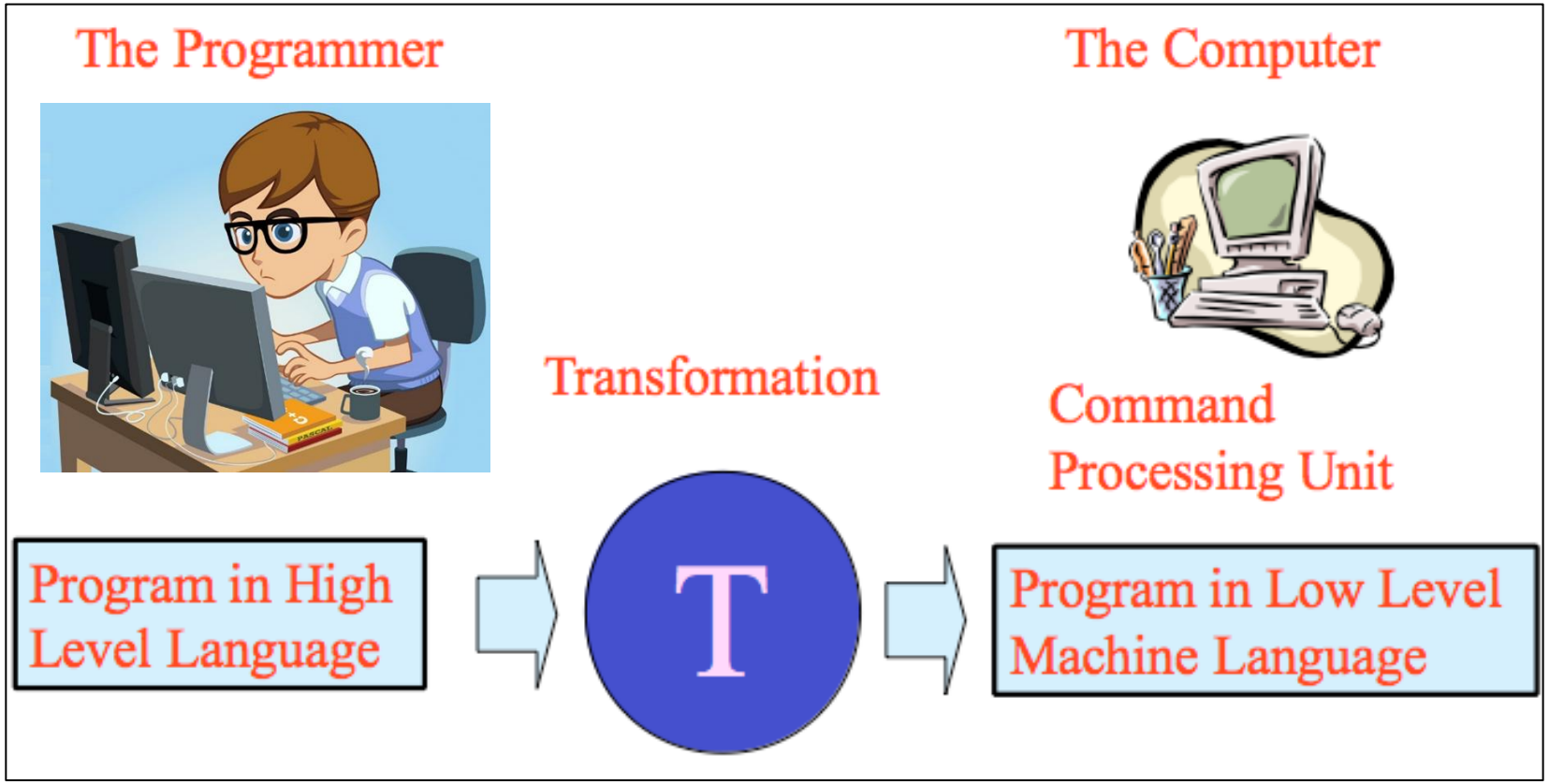


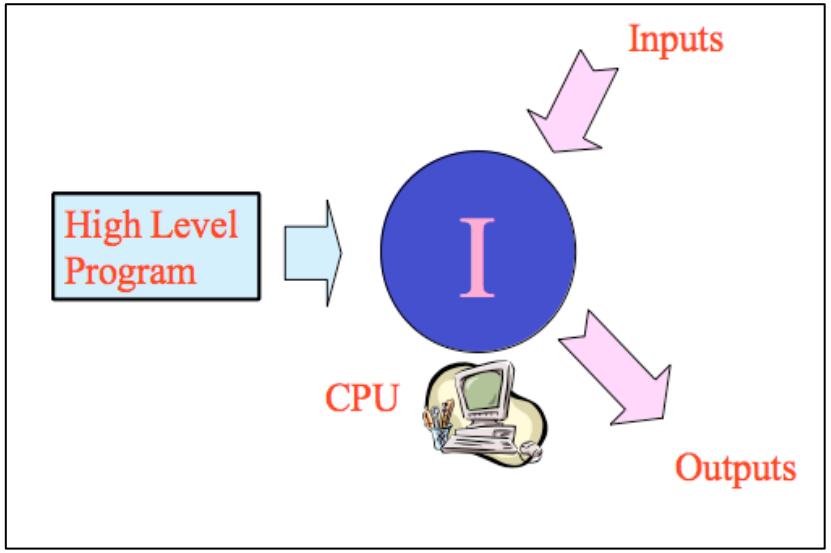
Over 500 different computer languages are listed by Wikipedia

Machine code (language)

- Computers understand only machine language
  - Basically, looks like a sequence of 1's and 0's
  - Very inconvenient to work with and non-intuitive
- All other computer languages were created for human convenience ("high-level")
- The computer does not understand C/Python/Java - They must be "translated" into machine language
- Other courses will teach you how hardware and software work.

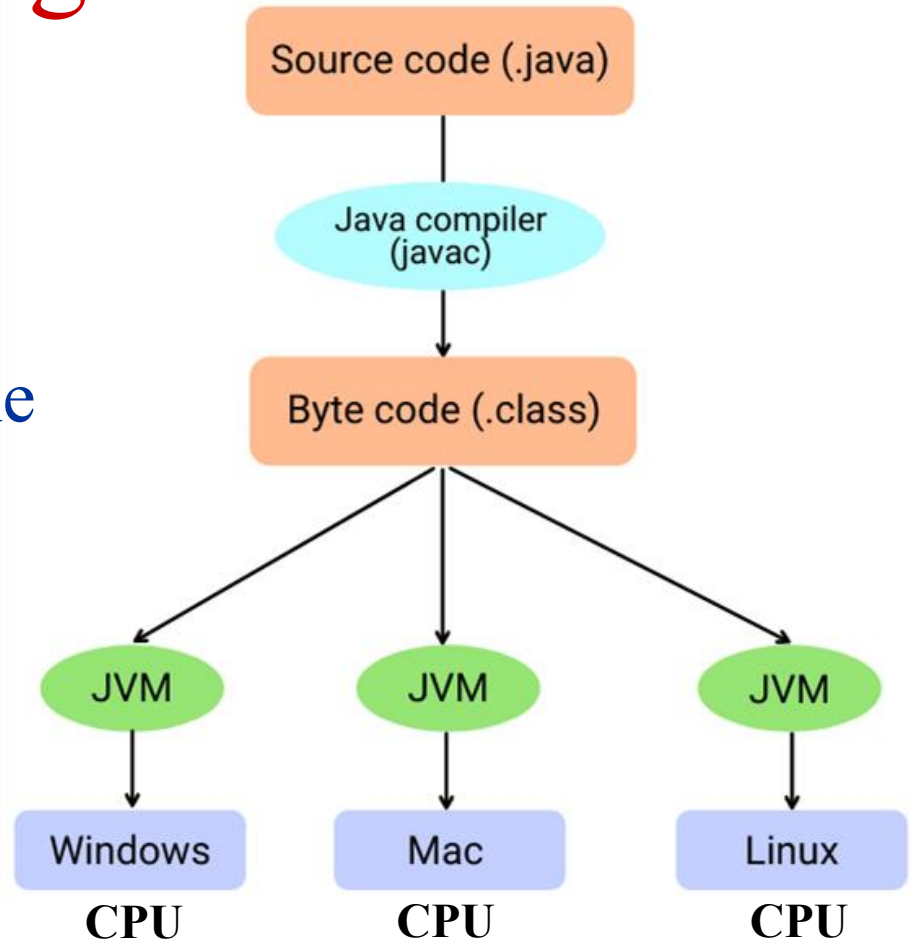
High level to machine level languages





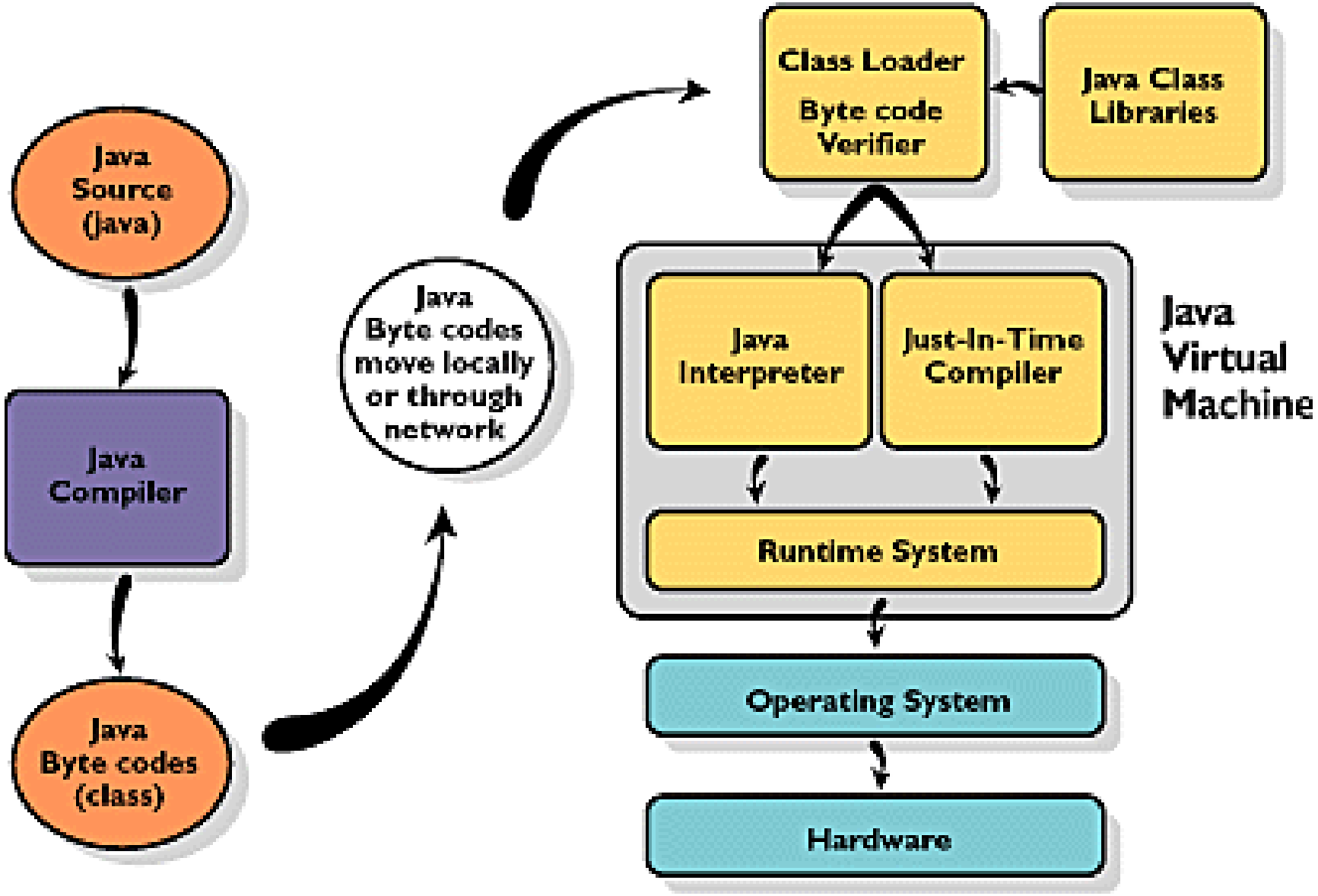
Executing a Java program

- Java uses both a compiler (מהדר) and an interpreter (מפרש).
- **Compilation** (הידור) is a translation of the entire high level programming language code to machine language called Java byte-code
- The JVM (Java virtual machine) **Interpretation** process (פירוש) is a line by line translation and execution performed on the CPU.



- A Central Processing Unit (CPU) is the "brain" of a computer, an essential hardware component that executes instructions from software and performs calculations, logic, and I/O operations

Java Development and Runtime Environment



Running Java

- Requires multiple software.
- You will run Java locally (on your pc).
- If your having troubles you can work online using an online Java compiler and debugger.

Java Installation

- ✓ JDK installation (version 25) - follow instruction on course website using Oracle java downloads
- ✓ CS: IntelliJ (community edition!) installation - create a project, a class and code away.. OR
- ✓ IEM: eclipse IDE for java developers installation instruction- create a project, a class and code away..
- ✓ Online JDB - Java compiler and debugger

JDK Installation

oracle.com/il-en/java/technologies/downloads/

ORACLE Search

Java Downloads

Java downloads | Tools and resources | Java Verified Portfolio | Java archive

Watch the JavaOne 2026 keynotes and s

Looking for other Java downloads?

Java 26, Java 25, Java 21, and earlier versi…ow

JDK 26 is the latest release of the Java SE Platform.

JDK 25 is the latest *Long-Term Support (LT…* …ava SE Platform.

JDK 21 is the previous Long-T… …release of the Java SE Platform.

Earlier JDK versions … …low.

JDK 26 | JDK 25 | JDK 21

Java SE Development Kit 26.0.2.1 downloads

JDK 26 binaries are free to use in production and free to redistribute, … cost, under th…

JDK 26 will receive updates under these terms, u… …t will be supe…

Linux | macOS | Windows

JDK 25 | JDK 21

Java SE Development Kit 21.0.8 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms an…

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subseq…
(OTN) and production use beyond the limited free grants of the OTN license will require a fee.

Learn about Java SE Subscription

Linux | macOS | Windows

Product/file description	File size	Download
x64 Compressed Archive	186.05 MB	https://download.oracle.com/jav…
x64 Installer	164.42 MB	https://download.oracle.com/jav…
x64 MSI Installer	163.16 MB	https://downloa…racle.com/jav…

IntelliJ IDEA Installation

jetbrains.com/idea/download/?section=windows

JETBRAINS AI Developer Tools Team Tools Education Solutions Support Store

IntelliJ IDEA JetBrains IDEs Coming in 2025.3 What's New Features Resources Pricing Download

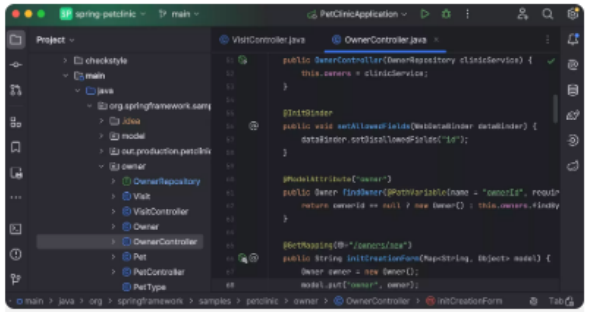
Windows macOS Linux

IntelliJ IDEA Ultimate

The Leading IDE for Professional Development in Java and Kotlin

Download .exe (Windows)

Free 30-day trial



Version: 2025.2.3
Build: 252.26830.84
2 October 2025

System requirements Other versions
Installation instructions Third-party software

We're committed to giving back to our wonderful community, which is why IntelliJ IDEA Community Edition is completely free to use

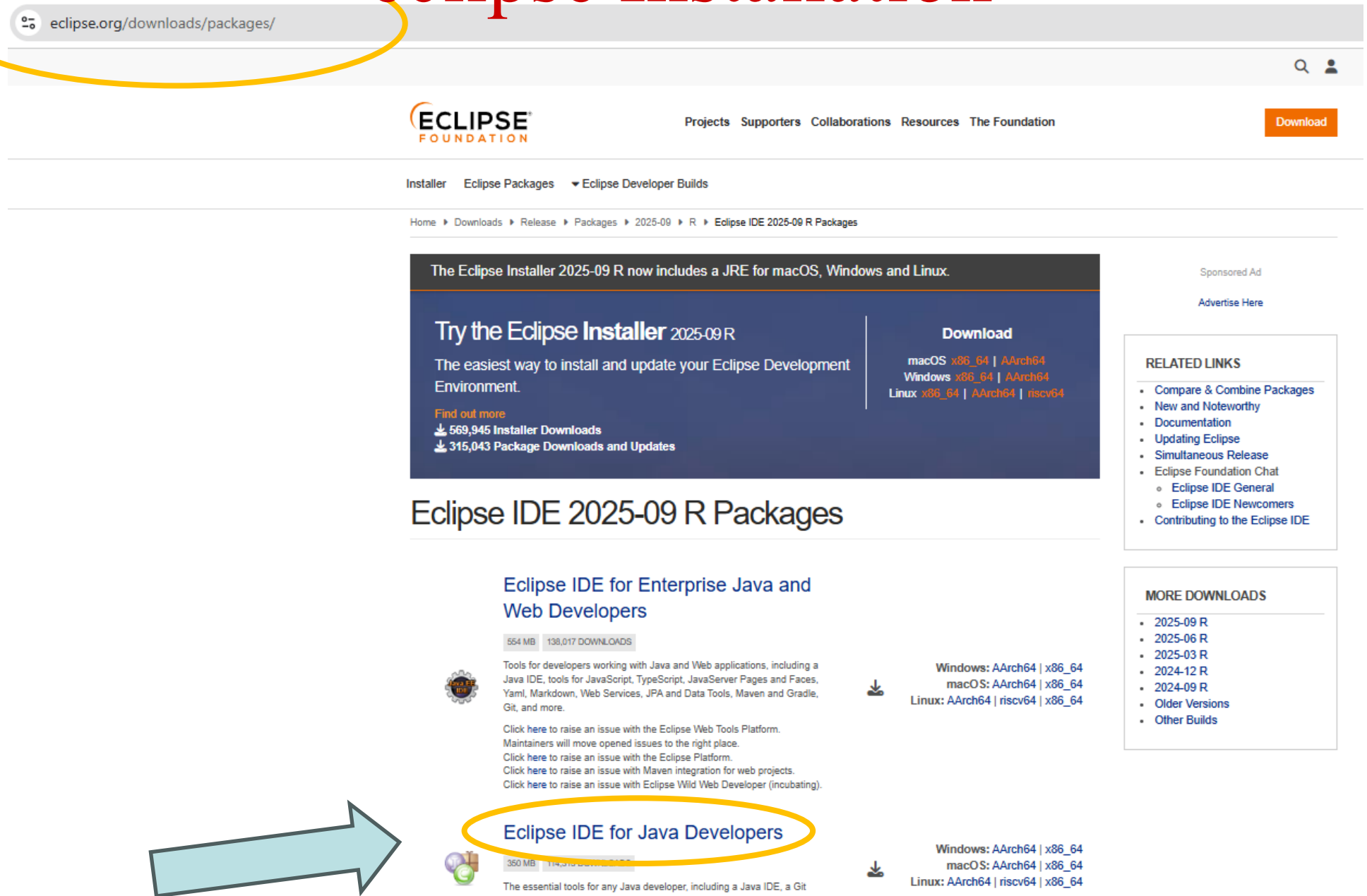
IntelliJ IDEA Community Edition

The IDE for Java and Kotlin enthusiasts

Download .exe (Windows)

Free, built on open source

eclipse Installation



So far…

- We are learning **How to program** (not exclusively in Java!)
- Computer program
- Java compiler and interpreter

My first Java program

```java
public class Main {

    public static void main(String[] args) {

        System.out.println("Hello world!");

    }

}
```

My first Java program

```
1  public class Main {
2
3      public static void main(String[] args) {
4
5          System.out.println("Hello world!");
6
7      }
8
9  }
```

Variables

Using variables to store data

- A variable is a named storage location in a computer's memory that holds a value.
- Each variable has:
  - A value (content, the stored data)
  - A name (a shortcut to its address in memory)
  - A type (int, double, boolean, char)

Variables (משתנים) in Java

- A variable is a named storage location in a computer's memory that holds a value.
- A variable must be **declared** (only once):

```
<type>  <variable name>;
int myNumber;
```

name	type	value
myNumber	int	

- A variable must be **assigned** a value before use:

```
<variable name> = <value>;
myNumber = 24;
```

name	type	value
myNumber	int	24

- We can do both at one statement:

```
<type> <variable name> = <value>;
int primeNumber = 11;
```

name	type	value
myNumber	int	24
primeNumber	int	11

Variables – Model View (טבלת המשתנים)

- Some more examples – Introducing the Model view

```
<type>  <variable name> = <value>;
```

The assignment operator

a value(literal) (on the right side) is assigned to a variable (on the left side).

```
int myNumber = 24;
int primeNumber = 11;
```

name	type	value
myNumber	int	24
primeNumber	int	11

Variables – Model View (טבלת המשתנים)

- Some more examples – Introducing the Model view

```
<type>  <variable name> = <value>;
```

The assignment operator

a value/literal (on the right side) is assigned to a variable (on the left side).

```
double pi = 3.14;
boolean isPrime = true;
char myChar = 'b';
```

name	type	value
?	?	?
?	?	?
?	?	?

Primitive Data types (טיפוסים פרימיטיביים) in Java

- 8 primitive types
  - Whole numbers: long, int, short, byte (8, 16, 32, 64 bits)



  - Real numbers: double, float (4,8 bytes - 32, 64 bits)
  - True/False: boolean (1 byte – 8 bits)
  - chars (every char has a numeric representation) (2 bytes - 16 bits)
- Each tells the computer what kind of information a variable will hold.
- A maximal and minimal values (in cyclic order)

ASCII table

- Encoding letters from assigned numbers

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

ASCII table

The **ASCII table** (American Standard Code for Information Interchange) is a character encoding standard that represents text in computers and other devices that handle text.

ASCII assigns a unique number (code) to each character, which allows computers to understand and display letters, numbers, punctuation marks, and control characters consistently across different platforms.

Why do we need variables?

- Computer programs manipulate data
- Data is given as input or calculated during the program execution
- To be later accessed, variables must be remembered

Assignment - simplified

The command:	Is like telling java:
>> int x = 8	Get a box, name it ‘x’ **Assign** 8 to it.
>> double y = 12.1	Get a box, name it ‘y’ **Assign** 12.1 to it.
>> y = y + 1;	Fetch the value of ‘y’, add 1 to that value (result is stored in a temporary storage) . now assign the sum to variable ‘y’.

Variables and assignments

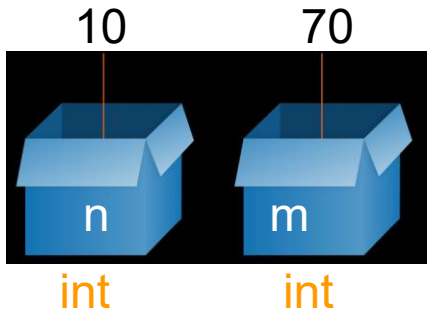
```
int n = 10;
int m = (10 + 4) * 5;
```

Left-hand side is a variable

Right-hand side is an expression

Java:

1. Evaluates the **expression**
2. Assigns its value to the **variable**



Java convention:

variable's name - a sequence of letters and digits, starting with a small letter:

Examples: ans, res, temp, n, sum, sumOfGrades, dayOfWeek

Assignment and Model View

- First, evaluate the **expression**. Then assign it's value to the **variable**.

```
1.   int i = 0;
2.   int j = i;
3.   i = i + 1;
4.   int k;
5.   k = 2;
6.   j = (k + i) * 2;
```

name	type	value

Why do we need different types?

- In java, like other statically typed languages (C, C++, C#), data types are essential for:
  - **Type safety** - Java prevents you from performing illogical or invalid operations.
  - **Efficiency and Performance** - When the compiler knows each variable's type, it can allocate the exact amount of memory needed and optimize the code.
  - **Error Detection at Compile Time** - Because Java is statically typed, many errors are caught during compilation rather than at runtime — saving debugging time later.

So far…

- We are learning **How to program** (not exclusively in Java!)
- Computer program
- Java compiler and interpreter

- We use **variables** and **assignments** to store information in our programs

Undeclared variables (compilation error)

```
1.    int i = 0;
2.    int i = k;
// error !!!
```

name	type	value
i	int	0

Compilation failed due to following error(s).

```
Main.java:13: error: cannot find symbol
                i = k;
                    ^
  symbol:   variable k
  location: class Main
1 error
```

Unassigned variables (compilation error)

```
1.     int i;
2.     i = i + 1;
// error !!!
```

name	type	value
i	int	

Compilation failed due to following error(s).

```
Main.java:13: error: variable i might not have been initialized
                i = i + 1;
                    ^

1 error
```

More basics:
arithmetic and boolean operators,
boolean variables

Arithmetic operators

Operator	Use	Description
+	x + y	Sum of x and y
-	x - y	Subtracts y from x
*	x * y	Multiplies x by y
/	x / y	Divides x by y
%	x % y	Modulo: the remainder of dividing x by y

Playing with arithmetic operators…

Operations on numbers

Expression	Result
8%5	3
5%8	5
8/5	1 (int)
8/5.0	1.6 (double)
8.0/5.0	1.6 (double)
8.0/5.0	1.6 (double)

operator overloading

the + operator with strings

operator overloading!

```
System.out.println("My lucky number is 7");
System.out.println("My lucky number is " + 7);
System.out.println("My lucky number is " + 3 + 4);
System.out.println("My lucky number is " + (3 + 4));
System.out.println(3 + 4 + " is my lucky number");
```

```
My lucky number is 7
My lucky number is 7
My lucky number is 34
My lucky number is 7
7 is my lucky number
```

Boolean variables & Operators

- Either ***true*** or ***false*** (true and false are boolean literals):

  ✓ `boolean bool = false;`
  ✓ `boolean isPrime = true;`

- **Logical operators**:

  – “Unique” to Boolean variables.
  – Result is a Boolean value (either True or False)

Boolean (logical) operators

Operator		Description
a **&&** b	and	Both true: **true**, otherwise: **false**.
a **||** b	or	At least one is true: **true**, Otherwise: **false**.
! a	not	a is false: **true**, a is true: **false**

and

false	**true**	**&&**
false	true	**true**
false	false	**false**

or

false	**true**	**||**
true	true	**true**
false	true	**false**

not

	!
false	**true**
true	**false**

Logical operators: &&, ||, !

```
boolean a = true;
boolean b = true;
boolean c = false;
boolean d = false;
```

Expression	Value
`a && b`	?
`a && c`	
`a || c`	
`c || d`	
`!a`	

Logical operators: &&, ||, !

```
boolean a = true;
boolean b = true;
boolean c = false;
boolean d = false;
```

Expression	Value
`a && b`	`true`
`a && c`	`false`
`a || c`	`true`
`c || d`	`false`
`!a`	`false`

Comparison operators

Compares two numbers and returns a boolean literal

Operator	Name	Description
x < y	Less than	true if x is less than y, otherwise false.
x > y	Greater than	true if x is greater than y, otherwise false.
x <= y	Less than or equal to	true if x is less than or equal to y, otherwise false.
x >= y	Greater than or equal to	true if x is greater than or equal to y, otherwise false.
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

Comparison operators

Expression	Result
5 == 5	?
14 == 14.0	
6 != 2 * 3	
-2 >= 1	
3 <= 3	
3 > 3	
19.1 > 7	

Comparison operators

Expression	Result
5 == 5	true
14 == 14.0	true
6 != 2 * 3	false
-2 >= 1	false
3 <= 3	true
3 > 3	false
19.1 > 7	true

Comparison operators

Compares two booleans and returns a boolean literal

Operator	Name	Description
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

```
boolean isPrime = true;
```

Expression	Value
`isPrime`	?
`isPrime == true`	
`isPrime != isPrime`	
`!isPrime`	

Comparison operators

Compares two booleans and returns a boolean literal

Operator	Name	Description
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

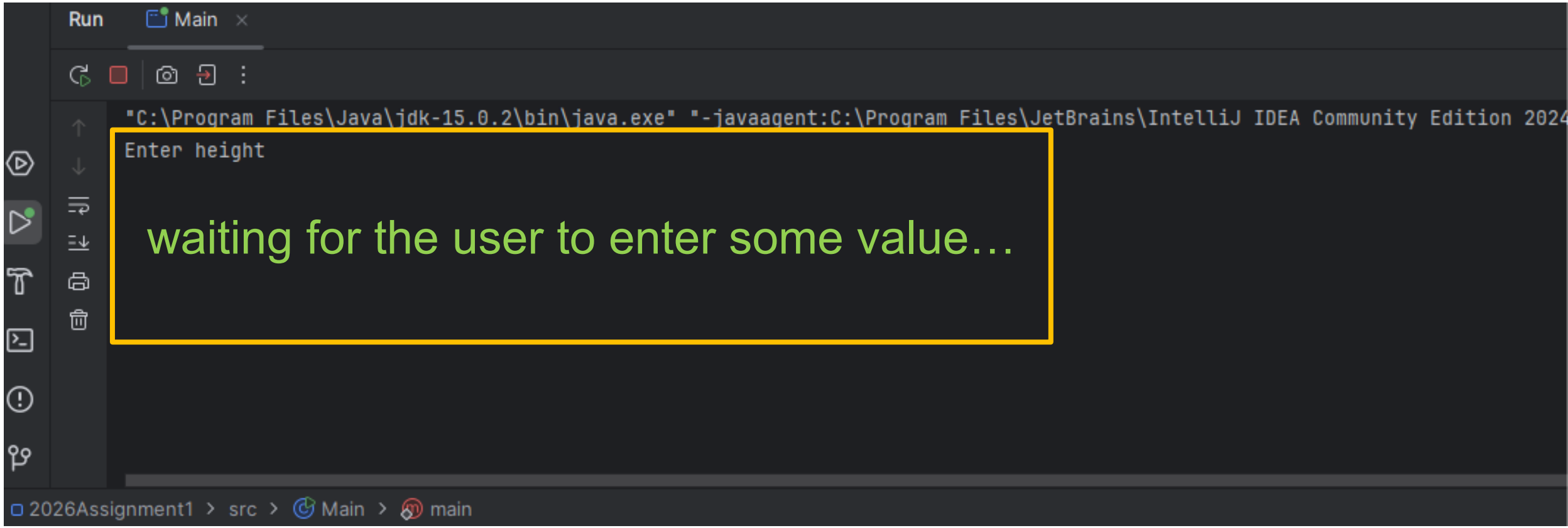
```
boolean isPrime = true;
```

Expression	Value
`isPrime`	`true`
`isPrime == true`	`true`
`isPrime != isPrime`	`false`
`!isPrime`	`false`

Input / Output

In a Java IDE (like **IntelliJ IDEA**, Eclipse, or NetBeans) –

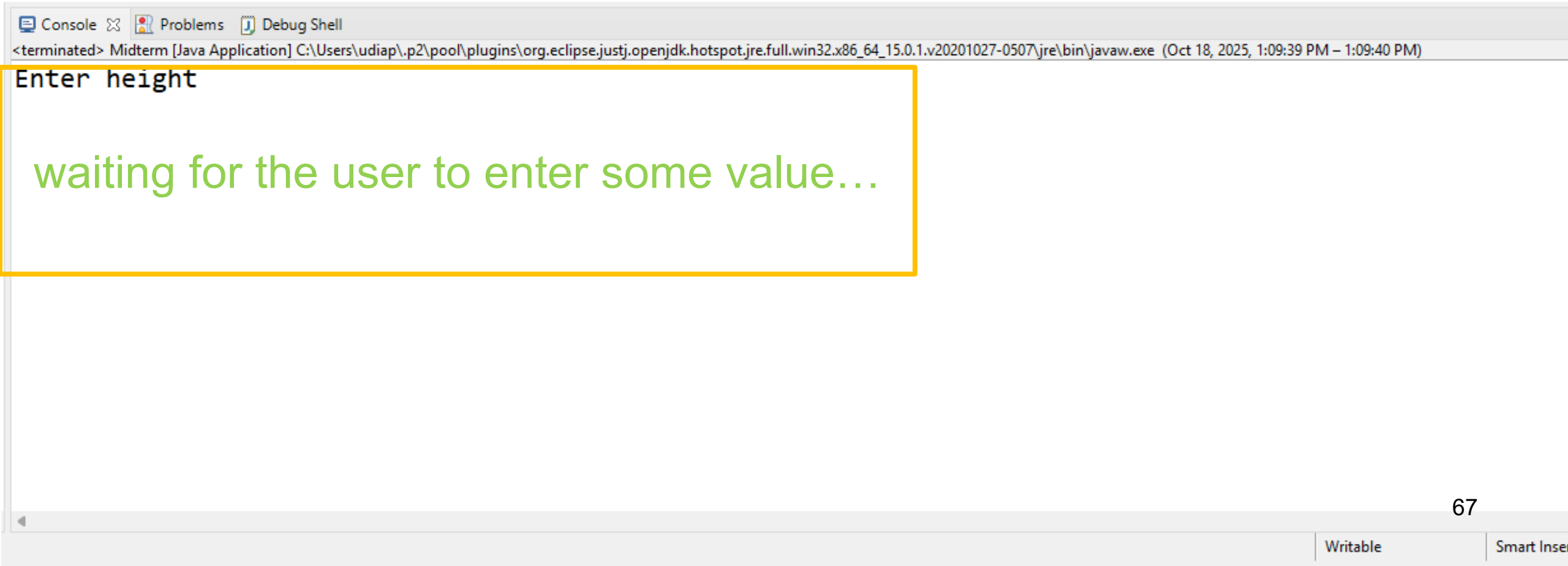
the **console** **is the text window where your program's input and output appear when you run it.**



Input / Output

In a Java IDE (like IntelliJ IDEA, **Eclipse**, or NetBeans) –

the **console** **is the text window where your program's input and output appear when you run it.**



Input / Output

Input: myScanner.nextInt();

Output:

System.out.print("Hello world");

System.out.println("Hello world");

```
1  import java.util.Scanner;
2  public class Main {
3
4      public static void main(String[] args) {
5
6          Scanner myScanner = new Scanner(System.in);
7          int n = myScanner.nextInt();
8          System.out.println(n);
9
10     }
11
12 }
```

import Scanner library containing all relevant data java requires

create a new "Scanner" object which can read input from the console

print the value of variable n to the console

Flow control

Should a program always execute line after line?

Different inputs >> different execution.

- For example: a user enters password, what to do if it's right? if it's wrong?

Structures that alter code linearity are -

- conditional statements
- loops
- functions

Conditional Statements

Block statement

A block of code contains one or more statements inside curly brackets:

```
{
    statement1
    statement2
}
```

```
{
        System.out.println("prime");
        System.out.println("hurray!");
}
```

Each block statement is indented one “tab”.

Conditional statements

Indentation:



- Following the if statement:
  - Open a new block by adding a left curly bracket
  - all statements within the block begin with one tab to the right!

- This is called **indentation** and indicates the commands within the scope of the current block

Scope

The **scope of a variable** is the part of the program where that variable can be **accessed or used**.

The scope of a variable is the block in which it was declared.

Conditional statement: if

Used to conditionally execute a block of code

Syntax

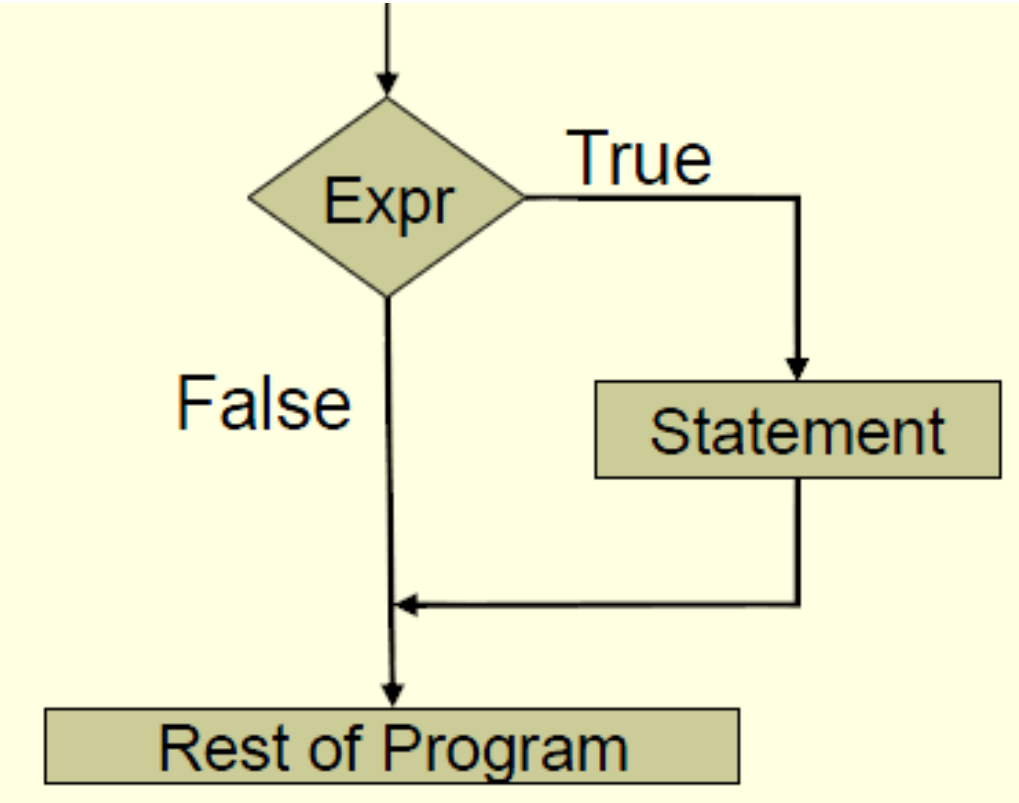
if *(expression)* **{**
    statement1
    statement2
}

```
if (isPrime) {
    System.out.println("prime");
    System.out.println("hurray!");
}
```

If *expression* is true, *statement* are executed.

if is a reserved word in Java

Conditional statements



Conditional statements - examples

```
11          int num = 360;
12          if (num % 18 == 0) {
13              System.out.println(num + " is devisable by 18");
14              int result = num / 18;
15              System.out.println(result + " * 18 = " + num);
16          }
17          System.out.println("Goodbye");
```

```
360 is devisable by 18
20 * 18 = 360
Goodbye
```

Conditional statement: if else

Used to conditionally execute one of two blocks of code

Syntax

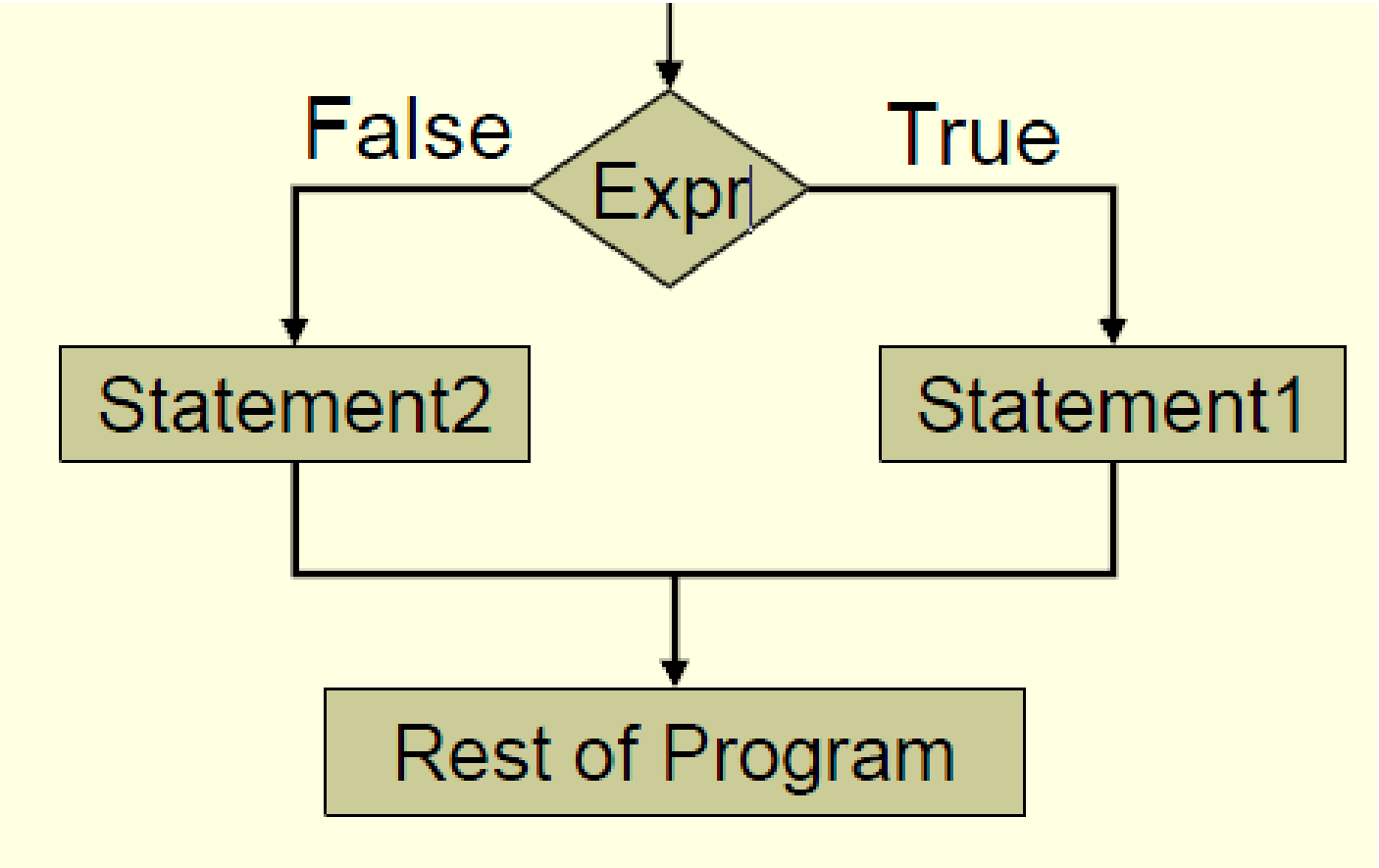
if *(expression)* **{**
    statement1
    statement2
}
else {
    statement3
    statement4
}

```
if (isPrime) {
    System.out.println("prime");
}
else {
    System.out.println("not prime");
}
```

If *expression* is true, the first block is executed. Else, the second block is executed.

else is a reserved word in Java

if-else statement



Example 1

```
if (2025 % 7 ==0) {
    System.out.println("2025 is devisable by 7");
}
else {
    System.out.println("2025 is not devisable by 7");
}
```

```
2025 is not devisable by 7
```

Example 2: building a square

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {

        Scanner myScanner = new Scanner(System.in);

        System.out.println("Enter height");
        int height = myScanner.nextInt();

        System.out.println("Enter width");
        int width = myScanner.nextInt();

        System.out.println("width, height = " + width + ", " + height);

        if (width == height) {
            System.out.println("It's a square!");
        }
        else{
            System.out.println("It's a rectangle!");
            width = height;
            System.out.println("width, height = " + width + ", " + height);
            System.out.println("And now it is a square :-)");
        }
    }
}
```

Read input from user

Display

Print output to console

Example 2: building a square

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {

        Scanner myScanner = new Scanner(System.in);

        System.out.println("Enter height");
        int height = myScanner.nextInt();

        System.out.println("Enter width");
        int width = myScanner.nextInt();

        System.out.println("width, height = " + width + ", " + height);

        if (width == height) {
            System.out.println("It's a square!");
        }
        else{
            System.out.println("It's a rectangle!");
            width = height;
            System.out.println("width, height = " + width + ", " + height);
            System.out.println("And now it is a square :-)");
        }
    }
}
```

Read input from user

Display

Print output to console

Example 2: building a square

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {

        Scanner myScanner = new Scanner(System.in);

        System.out.println("Enter height");
        int height = myScanner.nextInt();

        System.out.println("Enter width");
        int width = myScanner.nextInt();

        System.out.println("width, height = " + width + ", " + height);

        if (width == height) {
            System.out.println("It's a square!");
        }
        else{
            System.out.println("It's a rectangle!");
            width = height;
            System.out.println("width, height = " + width + ", " + height);
            System.out.println("And now it is a square :-)");
        }
    }
}
```

Read input from user

Display

Print output to console

"if" block

"else" block

Complex boolean expressions

Not actually complex!

- Logical operators transform multiple Boolean values to a single Boolean value (nothing new)
- Let's break it down to bits and pieces:

true if **at least one** is negative

```
(a <= 0 || b <= 0 || c <= 0)
```

true if a or b are negative **and** c is negative

```
(((a <= 0) || (b <= 0)) && c <= 0)
```

Nested conditional statements = conditional statements within conditional statements

Not actually complex!

- Every code block is “stand alone”.
- if / if-else statements can be placed within the block of an if/ if/else statement (nothing new) >>
- Allows for more sophisticated flow control!

- Let’s break it down to pieces

bits and pieces –
Example: expect a,b,c to be positive numbers

```
// Check that all edges are positive
if (a <= 0) {
    System.out.println("First edge must be positive!");
}
```

"if" block

bits and pieces –
Example: expect a,b,c to be positive numbers

```
// Check that all edges are positive
if (a <= 0) {
    System.out.println("First edge must be positive!");
}
else {
    if (b <= 0) {
            System.out.println ("Second edge must be positive!");
    }
    else {
        [redacted]
    }
}
```

"if" block

"else" block containing another if-else statement

bits and pieces –
Example: expect a,b,c to be positive numbers

```
// Check that all edges are positive
if (a <= 0) {
    System.out.println("First edge must be positive!");
}
else {
    if (b <= 0) {
            System.out.println ("Second edge must be positive!");
    }
    else {
            if (c <= 0) {
                    System.out.println("Third edge must be positive!");
            }
    }
}
```

"if" block

"else" block containing another if-else statement

"if" block

Example 3: triangle inequality

Read input from user

```
System.out.print("Enter first edge: ");
double a = myScanner.nextDouble();

System.out.print("Enter second edge: ");
double b = myScanner.nextDouble();

System.out.print("Enter third edge: ");
double c = myScanner.nextDouble();
```

Check input

```
// Check that all edges are positive
if (a <= 0 || b <= 0 || c <= 0) {
    System.out.println("All edges must be positive!");
}
```

Print output to console

```
// Triangle inequality rule:
// The sum of any two sides must be greater than the third.
else if (a + b >= c && a + c >= b && b + c >= a) {
    System.out.println("Yes, these edges can form a triangle.");
}
else {
    System.out.println("No, these edges cannot form a triangle.");
}
```

Example 3: triangle inequality

Read input from user

```
System.out.print("Enter first edge: ");
double a = myScanner.nextDouble();

System.out.print("Enter second edge: ");
double b = myScanner.nextDouble();

System.out.print("Enter third edge: ");
double c = myScanner.nextDouble();
```

Check input

```
// Check that all edges are positive
if (a <= 0 || b <= 0 || c <= 0) {
    System.out.println("All edges must be positive!");
}
```

"if" block

Print output to console

```
// Triangle inequality rule:
// The sum of any two sides must be greater than the third.
else if (a + b >= c && a + c >= b && b + c >= a) {
    System.out.println("Yes, these edges can form a triangle.");
}
else {
    System.out.println("No, these edges cannot form a triangle.");
}
```

Example 3: triangle inequality

Read input from user

```
System.out.print("Enter first edge: ");
double a = myScanner.nextDouble();

System.out.print("Enter second edge: ");
double b = myScanner.nextDouble();

System.out.print("Enter third edge: ");
double c = myScanner.nextDouble();
```

Check input

```
// Check that all edges are positive
if (a <= 0 || b <= 0 || c <= 0) {
    System.out.println("All edges must be positive!");
}
```

"if" block

Print output to console

```
// Triangle inequality rule:
// The sum of any two sides must be greater than the third.
else if (a + b >= c && a + c >= b && b + c >= a) {
    System.out.println("Yes, these edges can form a triangle.");
}
else {
    System.out.println("No, these edges cannot form a triangle.");
}
```

"else" block containing another if-else statement

Check list for today!

- Get into BGU email account
- Visit course web site (Moodle)
- Bring laptop to practical sessions! (Fully charged)

- See you tomorrow ☺



Hatchala - Introduction to computer science in Java - Fall 26'-27'

Department of Computer Science and Information

Ben-Gurion University of the Negev

Based on both Dr. Assaf Zaritsky slides for "Intro to CS" (IS) and slides for "Introduction to CS" (CS) courses at BGU

PS 1: First acquaintance with Java

My Name

- A little bit about my self..

A personal note on future HW

It will take you a **lot of time** and effort

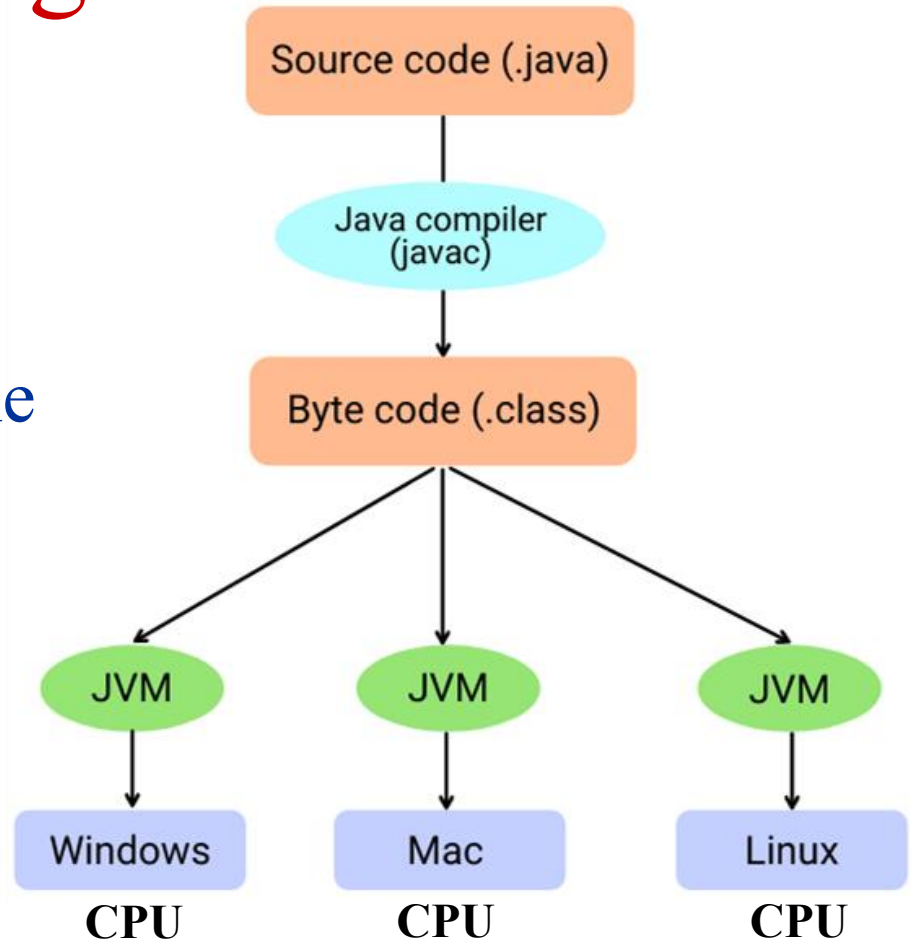
But

There is no other way to learn how to program



Executing a Java program

- Java uses both a compiler (מהדר) and an interpreter (מפרש).
- **Compilation** (הידור) is a translation of the entire high level programming language code to machine language called Java byte-code
- The JVM (Java virtual machine) **Interpretation** process (פירוש) is a line by line translation and execution performed on the CPU.



- A Central Processing Unit (CPU) is the "brain" of a computer, an essential hardware component that executes instructions from software and performs calculations, logic, and I/O operations

Running Java

- Requires multiple software.
- You will run Java locally (on your pc) / lab computer.
- If your having troubles you can work online using an online Java compiler and debugger.

Using variables to store data

- A variable is a named storage location in a computer's memory that holds a value.
- Each variable has:
  - A value (content, the stored data)
  - A name (a shortcut to its address in memory)
  - A type (int, double, boolean, char)

Assignment and Model View

- First, evaluate the **expression**. Then assign it's value to the **variable**.

```
1.  int i = 0;
2.  int j = i;
3.  i = i + 1;
4.  int k;
5.  k = 2;
6.  j = (k + i) * 2;
7.  char ch = 'm';
8.  boolean thisIsMyFirstTime = true;
```

name	type	value

Assignment and Model View

- First, evaluate the **expression**. Then assign it's value to the **variable**.

```
1.  int i = 0;
2.  int j = i;
3.  i = i + 1;
4.  int k;
5.  k = 2;
6.  j = (k + i) * 2;
7.  char ch = 'm';
8.  boolean thisIsMyFirstTime = true;
```

name	type	value
i	int	1
j	int	0
k	int	6
ch	char	'm'
thisIsMyFirstTime	boolean	true

ASCII table

- Encoding letters from assigned numbers

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

Variables – Model View (טבלת המשתנים)

- Some more examples – Introducing the Model view

  What does each different c mean?

```
char c = 'c';
System.out.println(c);
System.out.println('c');
System.out.println("c");

System.out.println(c + c);
```

name	type	value
c	char	‘c’

Undeclared variables (compilation error)

```
1.    int i = 0;
2.    int i = k;
// error !!!
```

name	type	value
i	int	0

Compilation failed due to following error(s).

```
Main.java:13: error: cannot find symbol
                i = k;
                    ^
  symbol:   variable k
  location: class Main
1 error
```

Unassigned variables (compilation error)

```
1.     int i;
2.     i = i + 1;
// error !!!
```

name	type	value
i	int	

Compilation failed due to following error(s).

```
Main.java:13: error: variable i might not have been initialized
                i = i + 1;
                    ^

1 error
```

Arithmetic operators

Operator	Use	Description
+	x + y	Sum of x and y
-	x - y	Subtracts y from x
*	x * y	Multiplies x by y
/	x / y	Divides x by y
%	x % y	Modulo: the remainder of dividing x by y

Playing with arithmetic operators…

the + operator with strings

operator overloading!

```
System.out.println("My lucky number is 7");
System.out.println("My lucky number is " + 7);
System.out.println("My lucky number is " + 3 + 4);
System.out.println("My lucky number is " + (3 + 4));
System.out.println(3 + 4 + " is my lucky number");
```

```
My lucky number is 7
My lucky number is 7
My lucky number is 34
My lucky number is 7
7 is my lucky number
```

Boolean variables & Operators

- Either ***true*** or ***false*** (true and false are boolean literals):

  ✓ `boolean bool = false;`
  ✓ `boolean isPrime = true;`

- **Logical operators**:

  – “Unique” to Boolean variables.
  – Result is a Boolean value (either True or False)

Boolean (logical) operators

Operator		Description
a **&&** b	and	Both true: **true**, otherwise: **false**.
a **||** b	or	At least one is true: **true**, Otherwise: **false**.
! a	not	a is false: **true**, a is true: **false**

and

false	**true**	**&&**
false	true	**true**
false	false	**false**

or

false	**true**	**||**
true	true	**true**
false	true	**false**

not

	!
false	**true**
true	**false**

Logical operators: &&, ||, !

```
boolean a = true;
boolean b = true;
boolean c = false;
boolean d = false;
```

Expression	Value
`a && b`	`true`
`a && c`	`false`
`a || c`	`true`
`c || d`	`false`
`!a`	`false`

Comparison operators

Compares two numbers and returns a boolean literal

Operator	Name	Description
x < y	Less than	true if x is less than y, otherwise false.
x > y	Greater than	true if x is greater than y, otherwise false.
x <= y	Less than or equal to	true if x is less than or equal to y, otherwise false.
x >= y	Greater than or equal to	true if x is greater than or equal to y, otherwise false.
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

Comparison operators

Expression	Result
5 == 5	true
14 == 14.0	true
6 != 2 * 3	false
-2 >= 1	false
3 <= 3	true
3 > 3	false
19.1 > 7	true

Comparison operators

Compares two booleans and returns a boolean literal

Operator	Name	Description
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

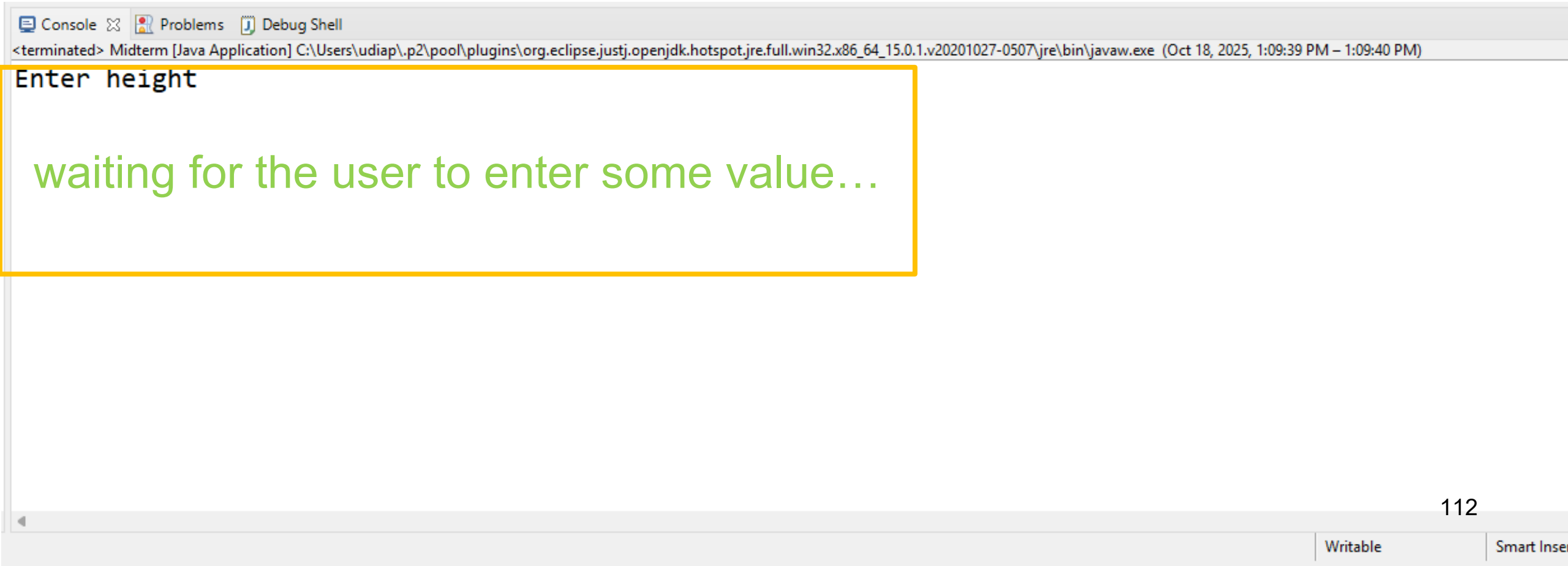
```
boolean isPrime = true;
```

Expression	Value
`isPrime`	`true`
`isPrime == true`	`true`
`isPrime != isPrime`	`false`
`!isPrime`	`false`

Input / Output

In a Java IDE (like IntelliJ IDEA, **Eclipse**, or NetBeans) –

the **console** **is the text window where your program's input and output appear when you run it.**



Input / Output

Input: myScanner.nextInt();

Output:

System.out.print("Hello world");

System.out.println("Hello world");

```
1  import java.util.Scanner;
2  public class Main {
3
4      public static void main(String[] args) {
5
6          Scanner myScanner = new Scanner(System.in);
7          int n = myScanner.nextInt();
8          System.out.println(n);
9
10     }
11
12 }
```

import Scanner library containing all relevant data java requires

create a new "Scanner" object which can read input from the console

print the value of variable n to the console

Conditional statement: if

Used to conditionally execute a block of code

Syntax

if *(expression)* **{**
    statement1
    statement2
}

```
if (isPrime) {
     System.out.println("prime");
     System.out.println("hurray!");
}
```

If *expression* is true, *statement* are executed.

if is a reserved word in Java

Conditional statement: if else

Used to conditionally execute one of two blocks of code

Syntax

```
if (expression) {
    statement1
    statement2
}
else {
    statement3
    statement4
}
```

```
if (isPrime) {
    System.out.println("prime");
}
else {
    System.out.println("not prime");
}
```

If *expression* is true, the first block is executed. Else, the second block is executed.

else is a reserved word in Java

Example 2 : building a square

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {

        Scanner myScanner = new Scanner(System.in);

        System.out.println("Enter height");
        int height = myScanner.nextInt();

        System.out.println("Enter width");
        int width = myScanner.nextInt();

        System.out.println("width, height = " + width + ", " + height);

        if (width == height) {
            System.out.println("It's a square!");
        }
        else{
            System.out.println("It's a rectangle!");
            width = height;
            System.out.println("width, height = " + width + ", " + height);
            System.out.println("And now it is a square :-)");
        }
    }
}
```

Read input from user

Display

Print output to console

"if" block

"else" block

Example 3: triangle inequality

```
System.out.print("Enter first edge: ");
double a = myScanner.nextDouble();

System.out.print("Enter second edge: ");
double b = myScanner.nextDouble();

System.out.print("Enter third edge: ");
double c = myScanner.nextDouble();
```

Read input from user

```
// Check that all edges are positive
if (a <= 0 || b <= 0 || c <= 0) {
    System.out.println("All edges must be positive!");
}
```

"if" block

Check input

```
// Triangle inequality rule:
// The sum of any two sides must be greater than the third.
else if (a + b >= c && a + c >= b && b + c >= a) {
    System.out.println("Yes, these edges can form a triangle.");
}
else {
    System.out.println("No, these edges cannot form a triangle.");
}
```

Print output to console

"else" block containing another if-else statement

Java Installation

- ✓ JDK installation (version 21!) - follow instruction on course website using Oracle java downloads
- ✓ CS: IntelliJ (community edition!) installation - create a project, a class and code away.. OR
- ✓ IEM: eclipse IDE for java developers installation instruction- create a project, a class and code away..
- ✓ Online JDB - Java compiler and debugger

JDK Installation

oracle.com/il-en/java/technologies/downloads/

ORACLE Search

Java Downloads

Java downloads | Tools and resources | Java Verified Portfolio | Java archive

Watch the JavaOne 2026 keynotes and s

Looking for other Java downloads?

Java 26, Java 25, Java 21, and earlier versi…ow

JDK 26 is the latest release of the Java SE Platform.

JDK 25 is the latest *Long-Term Support (LT…* …ava SE Platform.

JDK 21 is the previous Long-T… …release of the Java SE Platform.

Earlier JDK versions … …low.

JDK 26 | JDK 25 | JDK 21

Java SE Development Kit 26.0.2.1 downloads

JDK 26 binaries are free to use in production and free to redistribute, … cost, under th…

JDK 26 will receive updates under these terms, u… …t will be supe…

Linux | macOS | Windows

JDK 25 | JDK 21

Java SE Development Kit 21.0.8 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms an…

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subseq…
(OTN) and production use beyond the limited free grants of the OTN license will require a fee.

Learn about Java SE Subscription

Linux | macOS | Windows

Product/file description	File size	Download
x64 Compressed Archive	186.05 MB	https://download.oracle.com/jav…
x64 Installer	164.42 MB	https://download.oracle.com/jav…
x64 MSI Installer	163.16 MB	https://download.oracle.com/jav…

IntelliJ IDEA Installation

jetbrains.com/idea/download/?section=windows

JETBRAINS AI Developer Tools Team Tools Education Solutions Support Store

IntelliJ IDEA JetBrains IDEs Coming in 2025.3 What's New Features Resources Pricing Download

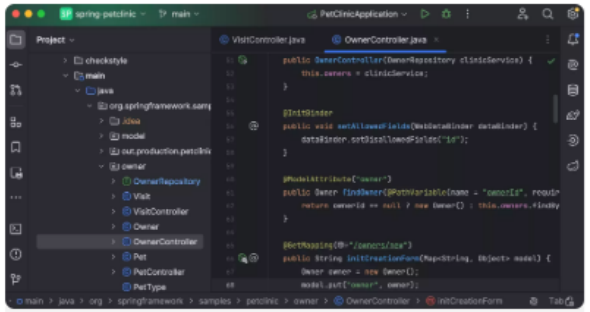
Windows macOS Linux

IntelliJ IDEA Ultimate

The Leading IDE for Professional Development in Java and Kotlin

Download .exe (Windows)

Free 30-day trial



Version: 2025.2.3
Build: 252.26830.84
2 October 2025

System requirements Other versions
Installation instructions Third-party software

We're committed to giving back to our wonderful community, which is why IntelliJ IDEA Community Edition is completely free to use

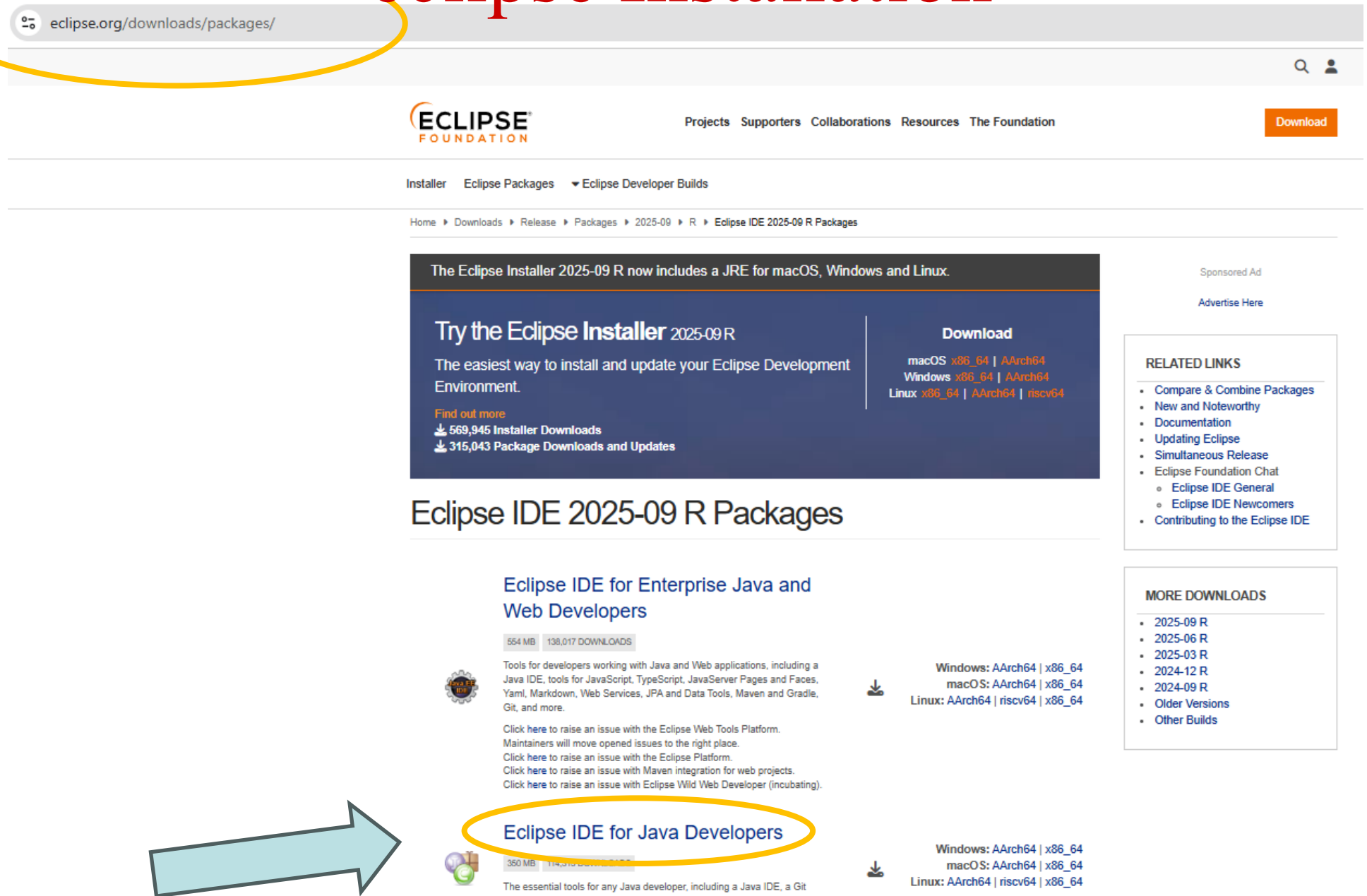
IntelliJ IDEA Community Edition

The IDE for Java and Kotlin enthusiasts

Download .exe (Windows)

Free, built on open source

eclipse Installation



Summary

Programming component	Examples
Output (printing)	System.out.println()/ System.out.print()
Input	scanner.nextInt() / scanner.nextDouble()
Variables declaration	int n; char c; double f; boolean b;
Value assignment for variables	int m = 5; f = 2.1;
Arithmetics	+, -, *, /, %
Logical operators	&&, ||, !
Comparing operators	<, >, <=, >=, ==, !=
Conditional statements	if, else